# MECH321 Python Installation and Setup Guide

***Python Control and scientific computing environment***

*Prof. Zengxi (Stephen) Pan · TYUT 2026*

**Course software. The current Project briefs use the supplied MATLAB scripts. This Python guide is optional support for alternative lecture demonstrations and independent checks. Follow each assessed brief's software and submission requirements.**

Use this guide only if you choose Python. An isolated virtual environment keeps the course packages separate from other Python installations.

## Required software

| Component | Use in MECH321 |
|---|---|
| Python 3 64 bit | Runs all course scripts |
| NumPy and SciPy | Numerical arrays, integration and scientific calculations |
| Matplotlib | Time response, Bode, Nyquist and comparison figures |
| python-control | Transfer functions, state space, feedback and stability tools |
| JupyterLab | Optional notebook interface for interactive class work |

## Recommended installation

Windows

- Install the current 64 bit Python 3 release from python.org. Select the option that adds Python to PATH if the installer shows it.
- Open PowerShell, change to the course-pack root folder (the folder containing Code), and run the commands below.

```
py -m venv Code\.venv
Code\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r Code\requirements.txt
python Code\verify_setup.py
```

macOS

- Install the current Python 3 package from python.org. Apple Silicon and Intel installers are both supported.
- Open Terminal, change to the course-pack root folder (the folder containing Code), and run the commands below.

```
python3 -m venv Code/.venv
source Code/.venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r Code/requirements.txt
python Code/verify_setup.py
```

## Expected verification result

The final command must print MECH321 Python setup passed, package version numbers and two finite closed-loop poles. A warning about Matplotlib building its font cache on the first run is harmless.

## Running the course examples

```
python Code/Python/L01_model_boundary.py
python Code/Python/L02_governing_equation.py
python Code/Python/L10_pi_saturation.py
jupyter lab
```

The ten revised lecture examples are paired by lecture number in Code/Python and Code/MATLAB. Python scripts write teaching figures to Code/Figures/Lecture; MATLAB scripts write them to Code/Figures/MATLAB. Students choosing MATLAB need MATLAB with Control System Toolbox. Run the scripts from the course-pack root folder. See Code/README.md for the complete current file map.

## Python Control command map

| Task | Python Control |
|---|---|
| Transfer function | ct.tf([numerator coefficients], [denominator coefficients]) |
| State space model | ct.ss(A, B, C, D) |
| Poles and zeros | ct.poles(system), ct.zeros(system) |
| Step response | ct.step_response(system) |
| Feedback connection | ct.feedback(forward, feedback) |
| Bode and Nyquist plots | ct.bode_plot(system), ct.nyquist_plot(system) |
| Gain and phase margins | ct.margin(system) |

## Troubleshooting

**Python command not found.** Close and reopen the terminal. On Windows, use py instead of python for environment creation.

**Activation is blocked in PowerShell.** Run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once, then activate the environment again.

**No module named control.** Activate Code/.venv, then rerun python -m pip install -r Code/requirements.txt.

**Jupyter uses the wrong environment.** Activate Code/.venv before running jupyter lab and confirm the notebook kernel reports the same Python path.

**Slycot is missing.** The core MECH321 examples do not require Slycot. Install it only if an advanced routine explicitly requests it.

## Official references

- Python virtual environments: https://docs.python.org/3/tutorial/venv.html
- Python Control documentation: https://python-control.readthedocs.io/
- SciPy installation: https://scipy.org/install/
- Jupyter installation: https://jupyter.org/install